SUPREME FACILITY MANAGEMENT LIMITED

TECHNICAL REQUIREMENTS DOCUMENT

Architecture • Technology • Security • Performance • Delivery

DRAFT FOR STAKEHOLDER REVIEW • VERSION 1.0 • 26 OCTOBER 2026

Document status and usage

Detailed planning baseline based on public website observations and project context. Not yet approved by the company. Confirm assumptions, scope, legal content, vendors, data handling and operational ownership with relevant stakeholders before production.

1. Technical Summary and Principles

Recommended baseline: Next.js App Router + React + TypeScript + Tailwind CSS; reusable accessible components; headless CMS only if editorial self-service is confirmed; managed hosting/CDN/WAF; approved server-side form delivery.

Architecture principle: Content-first, mostly statically rendered website. Use server rendering only when necessary. Keep public frontend, editorial system and any custom database appropriately separated.

Database decision: Do not add PostgreSQL/Supabase or MongoDB solely because the developer knows them. Add a database only for an approved workflow requiring structured records, permissions, retention, querying or integrations.

Security principle: Managed services where practical, least privilege, environment separation, automated checks, protected production deployments, tested recovery.

2. Proposed Stack

Framework: Next.js App Router, React and TypeScript. Server Components by default; Client Components only for browser interaction.

UI/styling: Tailwind CSS; accessible primitives such as Radix UI/shadcn/ui where useful; CSS variables/design tokens. Avoid overlapping styling systems.

Forms: React Hook Form may improve client UX; Zod or equivalent schema validation must also run server-side. Client validation is never a security boundary.

CMS: Sanity is a candidate if editors need self-service. Evaluate MFA/SSO, roles, preview, audit logs, export/backup, data terms/region, cost and company vendor policy. Compare approved alternatives if self-hosting is required.

Database/storage: Optional managed PostgreSQL/Supabase only for approved custom workflows. Use CMS/object storage for assets where appropriate. Never expose service-role credentials in client code.

Hosting/security edge: Company-approved managed Next.js platform or cloud; TLS, staging, rollback, CDN, logs, access controls and agreed data region. WAF/rate limiting at trusted edge where available.

Integrations/quality: Approved email/CRM endpoint; Playwright E2E; Vitest for suitable unit tests; axe-core/equivalent; Sentry or approved error monitoring; uptime, dependency and secret scanning.

3. Logical Architecture and Data Flows

Public request: Browser → DNS/CDN/WAF/TLS → Next.js routes/rendering → CMS read API and approved server-side integrations → HTML/assets. Cache public content at CDN when safe.

Editorial flow: Authorized editor → CMS identity/role controls → draft → preview → review/approval if required → publish → webhook/revalidation or scheduled rebuild → public page.

Form flow: Browser → HTTPS server endpoint/action → schema validation → bot/rate-limit checks → safe processing → approved CRM/email destination → success/error response. Never send privileged credentials from browser.

Investor document flow: Authorized publisher → approved storage/CMS → type/size/metadata validation → company approval → public metadata/download. Public files read-only; no anonymous upload.

Observability: Application/platform logs and metrics → restricted monitoring → alerts and incident workflow. Redact tokens, cookies, full form payloads and unnecessary personal data.

4. Repository and Content Architecture

Suggested structure: src/app routes/layouts; src/components/layout for header/footer; src/components/ui primitives; src/components/sections; src/lib integrations/validation/security; src/config public configuration; src/types shared types; public approved assets.

Code conventions: Strict TypeScript; clear naming; small components; accessible semantics; consistent formatting; explicit server/client boundaries. Avoid abstractions before repetition is demonstrated.

Routes: Stable readable slugs; page-specific metadata; reusable templates for service, sector, article, leadership and investor pages.

Content model: Use typed CMS schemas or typed data. Avoid scattering business copy and metrics in JSX. Store approved statistics and contacts in managed content.

Configuration: Environment-specific config; validate required environment variables; .env.example contains names and safe placeholders only.

5. Rendering, Cache and Performance

Rendering: Prefer static generation/ISR for public content. Use dynamic rendering only for genuine per-request freshness or request-specific behavior.

Cache policy: Define freshness by content type. Vacancies and investor disclosures must update within an agreed window. Publish actions must reliably trigger revalidation; never allow accidental stale disclosure content.

Media: Responsive images, modern formats, explicit dimensions, lazy-load below fold, prioritize LCP hero. Optimize local fonts and limit weights; confirm image/font licensing.

Client JS: Server Components by default; keep bundles small; load animation code only where needed; respect prefers-reduced-motion.

Targets: Core Web Vitals 'good' thresholds at 75th percentile where field data exists: LCP ≤2.5s, INP ≤200ms, CLS ≤0.1. Document lab versus field measurements.

6. Security Threat Model and Controls

Threats: Unauthorized publishing; compromised editor; form spam; injection/XSS; CSRF where relevant; malicious upload; dependency compromise; secret leakage; data exposure; denial of service; incorrect/stale investor disclosure; open redirects; cache misconfiguration.

Identity: Company-approved identity; MFA where supported; unique accounts; least privilege; separate editor/approver/admin if needed; prompt revocation after role change/offboarding.

Validation: Validate all untrusted input on server with allowlists/schemas; normalize and limit length/type; safely render user text; parameterized database queries; context-aware output encoding.

Sessions/CSRF: For authenticated cookie mutations, use framework-appropriate CSRF protection and origin checks as appropriate. Secure, HttpOnly and appropriate SameSite cookies. Never put privileged auth tokens in localStorage.

Headers: CSP based on actual integrations; test report-only then enforce as appropriate; frame-ancestors, X-Content-Type-Options, Referrer-Policy and Permissions-Policy. Enable HSTS after HTTPS readiness is confirmed.

Rate limits: Server/edge-enforced limits on public forms and endpoints; tune for legitimate corporate NATs; add bot challenge only when justified.

File handling: Authorized upload only; extension/MIME validation; size limits; safe generated object names; malware scanning where available; non-executable storage; least privilege; no public upload endpoints without approved need.

Secrets/supply chain: Managed secret store/environment variables; no .env in source control; no NEXT_PUBLIC_ for secrets; rotate exposed credentials; dependency/secret scanning; lockfiles; protected branch and reviewed CI/CD.

Logging/incident: Log relevant events with correlation IDs; restrict access and retention; redact personal data/secrets. Define alerting, incident response, credential rotation, restore and communication process.

Backup/security review: Agree backup scope, RPO/RTO and restore cadence; test restore. Perform threat-model review, dependency/config/access-control review and authorized security assessment before launch.

7. CMS and Content Model (Conditional)

Types: Global settings; navigation; homepage; service; sector; leadership profile; company page; article/news; vacancy; investor document metadata; office/contact; legal page.

Investor fields: Title, category, company/entity, reporting period, publication/document date, file reference, original URL if applicable, language, status, reviewer/approver and published timestamp.

Workflow: Draft → In Review → Approved → Published → Archived/Corrected. Actual states and segregation of duties set by Investor Relations/Compliance.

Permissions: Marketing owns marketing content; HR owns vacancies; Investor Relations/Company Secretary owns investor records; authorized approver publishes regulated disclosures; technical admin manages configuration. Confirm actual roles.

Preview/audit: Preview links protected, non-indexable and non-guessable. Drafts must not enter public cache/sitemap. Record who changed/published content and when if CMS supports it.

8. Data, Forms and Integrations

Data minimization: Avoid custom DB for brochure pages. Inquiry data is personal information and must go only to approved systems, with defined retention and access.

CRM/email: Endpoint/auth method TBD. Use server-side calls, timeouts, safe retries/idempotency where supported and generic public error messages. Do not expose API keys.

Recruitment: Confirm email-based applications vs recruitment platform vs new workflow. Do not collect CVs or identity documents before HR/Legal approve handling.

Analytics: Company-approved tool; define consent, event schema, retention and access. Never send names, emails, CVs or free-text form data to analytics.

Third-party inventory: Assess maps, video, social feeds, fonts, CAPTCHA, analytics and embedded content for availability, privacy, CSP and fallback behavior.

9. SEO, Redirects and Search

Metadata/crawlability: Unique titles/descriptions, canonical URLs, Open Graph, H1 hierarchy, accurate structured data, sitemap and intentional robots.txt. Noindex staging/preview and avoid duplicate filter URLs.

Redirects: Review every old URL mapping; 301 permanent moves directly to equivalent destination; test loops/chains and deep links.

Investor files: Preserve public document paths where feasible; redirect changed links to exact equivalent; verify external references.

Search: If required, define content types, ranking, filters, PDF indexing and no-results UX. Otherwise use clear navigation and focused investor filtering.

10. Accessibility, Testing and Release Gates

Accessibility: Target WCAG 2.2 AA; semantic landmarks; logical headings; keyboard menus/dialogs; visible focus; contrast; alt text; form labels/error summary; reduced motion; descriptive link/button semantics.

Automated/manual QA: Unit/component tests where valuable; Playwright journeys for navigation, mobile menu, contact form, careers, investor filters/download, 404 and redirects; keyboard/screen reader spot checks; current desktop/mobile browsers.

Content QA: Broken links; missing alt; duplicate/missing metadata; incorrect dates; expired roles; investor document count/metadata mismatch.

Release gates: Build/lint/type checks pass; critical E2E pass; no unresolved critical/high issues unless formally risk-accepted; content/design sign-off; investor reconciliation; redirects/forms verified; security and performance reviewed; backup/rollback tested; monitoring active.

11. Environments and Operations

Environments: Local, staging/preview and production. Separate credentials and access; never copy production personal data into development without approved sanitization.

Branch/deploy: Feature branches and PRs; protected main; automated checks; reviewed immutable builds; managed secrets; smoke tests and rollback.

Cutover: No production DNS changes until approval. Use approved cutover plan; verify TLS, redirects, form delivery, investor links and monitoring; retain rollback path.

Ownership: Company must assign domain/DNS, hosting/billing, CMS, alerts, dependency updates, content governance and emergency response ownership after internship handover.

12. Build Plan and AI Agent Contract

Stages: 0 discovery; 1 repo/CI/design tokens; 2 shared header/footer/components; 3 corporate pages; 4 CMS/dynamic content after decision; 5 forms/integrations; 6 content migration/redirects; 7 QA/security/performance; 8 launch/handover.

AI source of truth: Treat approved PRD/TRD as baseline. For TBD items, ask or use clearly labeled placeholders; never invent company facts, secrets, legal text, metrics or integrations.

Before coding: Inspect repository and conventions; propose short plan and dependencies before large changes.

Implementation: TypeScript; Server Components by default; accessible semantics; reusable components; server validation; no client secrets; no unapproved third-party service; no fabricated content.

Change reporting: Small reviewable changes; update tests/docs; report files changed, commands run, test results and risks. Never disable security checks just to pass a build.

Definition of done: Feature renders and works responsively; error states, accessibility, SEO, data handling and risk-appropriate tests are verified and documented.

13. Open Technical Decisions and Standards

Company decisions needed: CMS/vendor; hosting/cloud standard and data region; DNS/domain owner; CRM/email provider; recruitment flow; analytics/consent; document storage; SSO/MFA; backup RPO/RTO; uptime/SLA; monitoring; maintenance owner.

References: Next.js: https://nextjs.org/docs ; OWASP Top 10: https://owasp.org/www-project-top-ten/ ; WCAG 2.2: https://www.w3.org/TR/WCAG22/ ; Core Web Vitals: https://web.dev/articles/vitals .

Status: Proposed architecture only. IT/Security must approve the production stack, access model, hosting, data handling and operational controls.

Approval record

Role	Name	Decision/comments	Date
Executive sponsor			
Marketing			
HR			
Investor Relations / Compliance			
IT / Security			
Legal (if required)			